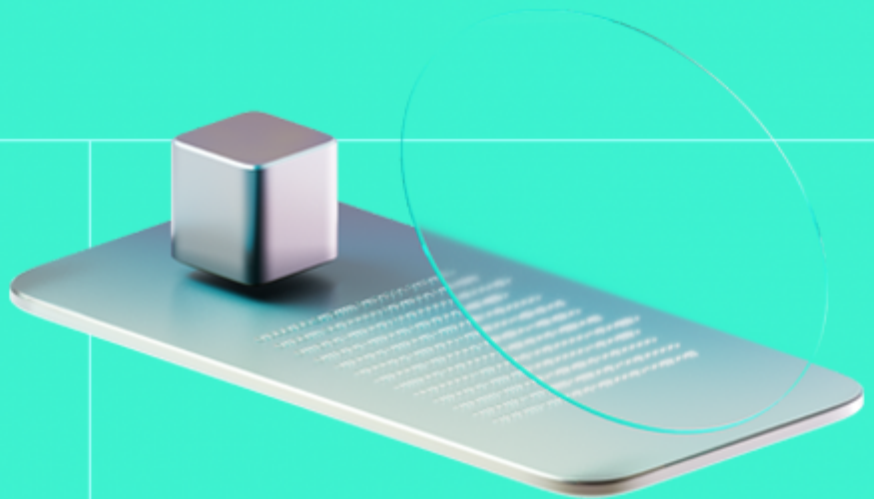




Smart Contract Code Review And Security Analysis Report

Customer: Haycen

Date: 04/03/2026



We express our gratitude to the Haycen team for the collaborative engagement that enabled the execution of this Smart Contract Security Assessment.

Haycen is a yield reward distribution system where tokens are deposited and allocated to recipients who can claim them at their leisure.

Document

Name	Smart Contract Code Review and Security Analysis Report for Haycen
Audited By	Seher Saylik
Approved By	Kornel Światłowski
Website	https://www.haycen.com
Changelog	03/03/2026 - Preliminary Report
	04/03/2026 - Final Report
Platform	Ethereum Mainnet, Avalanche C-Chain
Language	Solidity
Tags	Centralized, Yield distribution
Methodology	https://docs.hacken.io/methodologies/smart-contracts

Review Scope

Repository	https://github.com/Haycen/yield-claim-smart-contract
Commit	6e5135d8c6781d150c5407a65fcce36b2a31775a
Retest	621c8df0d65a2a656c2ef5cbd2e993d9b777e322

Audit Summary

The system users should acknowledge all the risks summed up in the risks section of the report

2	1	1	0
Total Findings	Resolved	Accepted	Mitigated

Findings by Severity

Severity	Count
Critical	0
High	0
Medium	0
Low	1

Vulnerability	Severity Status
F-2026-15163 - Users With Existing Allocations Cannot Claim After Token Is Delisted	Low Accepted
F-2026-15164 - Unchecked Arithmetic Could Be Used To Save gas	Info Fixed

Documentation quality

- Functional requirements are provided, and NatSpec is sufficient.
- Technical description is provided.

Code quality

- The development environment is configured.

Test coverage

Code coverage of the project is **88.52%** (branch coverage).

- Deployment and basic user interactions are covered with tests.

Table of Contents

System Overview	6
Privileged Roles	6
Potential Risks	7
Findings	8
Vulnerability Details	8
Disclaimers	13
Appendix 1. Definitions	14
Severities	14
Potential Risks	14
Appendix 2. Scope	15
Appendix 3. Additional Valuables	16

System Overview

YieldClaim is a yield reward distribution system with the following contracts:

YieldClaim — Upgradeable (UUPS) contract that holds whitelisted ERC-20 tokens, tracks per-recipient allocations, and lets recipients claim. Tokens are deposited by a designated depositor; a yield master increases or decreases allocations per recipient. Recipients claim allocated amounts at any time; a withdrawer can withdraw unallocated tokens.

Privileged roles

- The **owner** of YieldClaim can upgrade the implementation (UUPS), pause and unpause, add/remove tokens, set the depositor, withdrawer, and yield master, and initiate or cancel a two-step ownership transfer; the owner is therefore able to change critical behavior and role addresses at will.
- The **depositor** is the only address that can deposit whitelisted tokens into the contract.
- The **withdrawer** is the only address that can withdraw unallocated tokens (for any token), and can thereby recover funds sent by mistake or left after a token is delisted.
- The **yield master** is the only address that can increase or decrease allocations (single or batch); who receives how much yield is determined off-chain and applied on-chain by this role, so the yield master has full control over allocation amounts and recipients within available balance.

Potential Risks

- Rebasing tokens must never be whitelisted for this system because their balances can change automatically without transfers, breaking the core invariant that recorded allocations remain covered by the contract's actual balance and potentially causing claims to revert and funds to become stuck—an incompatibility the client explicitly acknowledged and documented.
- The logic that decides who receives yield and how much is not implemented on-chain: the `YieldClaim` contract does not encode any rules, formulas, or constraints for allocations and only allows the YieldMaster role to call `increaseAllocation` / `decreaseAllocation` with arbitrary token, amount, and recipient. As a result, allocation and deallocation are fully centralized—whoever controls the `YieldMaster` can at any time assign or remove entitlements for any address and any amount (within contract balance limits), with no on-chain accountability, transparency, or checks beyond the contract's balance and whitelist, so users must trust the operator not to misuse this power.
- When a token is delisted, the protocol can still **reduce or remove users' allocations, while users cannot claim** and no new allocations can be added. Entitlements can therefore be decreased or zeroed with no way for users to withdraw until the token is re-whitelisted.
- The owner can pause the contract in a way that prevents users from claiming their allocated tokens for an indefinite period, effectively locking user funds.
- The project is fully or partially centralized, introducing single points of failure and control. This centralization can lead to vulnerabilities in decision-making and operational processes, making the system more susceptible to targeted attacks or manipulation.
- The digital contract architecture relies on administrative keys for critical operations. Centralized control over these keys presents a significant security risk, as compromise or misuse can lead to unauthorized actions or loss of funds.
- The project's contracts are upgradable, allowing the administrator to update the contract logic at any time. While this provides flexibility in addressing issues and evolving the project, it also introduces risks if upgrade processes are not properly managed or secured, potentially allowing for unauthorized changes that could compromise the project's integrity and security.
- The contract suite allows for immediate upgrades without a mandatory review or waiting period, increasing the risk of rapid deployment of malicious or flawed code, potentially compromising the system's integrity and user assets.

Findings

Vulnerability Details

F-2026-15163 - Users With Existing Allocations Cannot Claim After Token Is Delisted - Low

Description:

When a token is removed from the whitelist via `removeToken()`, users who already have an allocation for that token can no longer call `claim()` or `claimAll()`. Their allocated amounts remain recorded in contract storage, but claiming is blocked until the token is re-whitelisted. This prevents users who are already entitled to yield from withdrawing their allocation and can lock funds until the owner re-whitelists the token.

YieldClaim uses a whitelist to control which tokens can be deposited, allocated, and claimed. The owner can remove a token with `removeToken()`.

1. `removeToken()`:

```
function removeToken(address token) external onlyOwner {
    if (!_whitelistedTokens[token]) revert TokenNotInWhitelist();
    _whitelistedTokens[token] = false;
    emit TokenRemoved(token);
}
```

Only `_whitelistedTokens[token]` is set to false. All allocation state is left unchanged:

- `_balances[recipient][token]` unchanged
- `_totalAllocated[token]` unchanged

2. `claim()` and `claimAll()`:

Both functions enforce that the token is whitelisted before performing any balance or transfer logic. If the token is not whitelisted, the call reverts with `TokenNotWhitelisted`.

```
function claim(address token, uint256 amount) external whenNotPaused nonReentrant {
    if (!_whitelistedTokens[token]) revert TokenNotWhitelisted();
    if (0 == amount) revert ZeroAmount();
    uint256 balance = _balances[msg.sender][token];
    if (amount > balance) revert InsufficientBalance();
    // Checks-Effects-Interactions: update state before external call
```



```

_balances[msg.sender][token] = balance - amount;
_totalAllocated[token] -= amount;
IERC20(token).safeTransfer(msg.sender, amount);
emit Claim(msg.sender, token, amount);
}

```

As a result, after a token is delisted:

- Users with non-zero `_balances[recipient][token]` are still “entitled” to that yield in the sense that the contract still holds their allocation and the tokens, but these users cannot withdraw that entitlement: every call to `claim()` or `claimAll()` for that token reverts.
- Entitlements are effectively locked until the owner calls `addToken()` again.

So, from a user’s perspective, users who are already entitled to yield cannot withdraw when the token is removed from the whitelist, even though their allocation still exists, and the contract may still hold the tokens.

Assets:

- `contracts/YieldClaim.sol` [<https://github.com/Haycen/yield-claim-smart-contract>]

Status:

Accepted

Classification

Impact Rate:	3/5
Likelihood Rate:	2/5
Exploitability:	Independent
Complexity:	Simple
Severity:	Low

Recommendations

Remediation: Relax the whitelist check for `claim()` and `claimAll()` so that they allow claims for a token that is not whitelisted only when the caller has a non-zero allocation for that token.

Resolution: Accepted and no further changes were applied.

The Haycen team stated that this is by design; a token will only be removed from the whitelist once it has been agreed and communicated

that the token is no longer supported. Any unclaimed tokens at the point of de-listing are essentially surrendered.

[F-2026-15164](#) - Unchecked Arithmetic Could Be Used To Save gas - Info

Description:

In several allocation and claim paths, the math is already safe (amounts checked before subtraction), but Solidity's default checked arithmetic still runs and costs extra gas. Using unchecked where overflow/underflow is impossible would reduce gas with no behavior change.

1. Claim and claim all:

After enforcing `amount <= balance`, the updates to the user balance and total allocated cannot underflow. Wrapping those updates in unchecked would save gas.

2. Decrease allocation (single and batch):

Decrease only runs when `amount <= currentBalance`, and total allocated is at least the sum of balances, so the subtractions cannot underflow. Doing the balance and total-allocated updates (and in the batch, the totalDecrease accumulation and final total-allocated write) in unchecked blocks would save gas.

3. Transfer ownership:

When a pending owner exists, the current owner is used in two events and can be read twice from storage. Reading it once into a local and using it in both events would avoid one SLOAD.

Assets:

- contracts/YieldClaim.sol [<https://github.com/Haycen/yield-claim-smart-contract>]

Status:

Fixed

Classification

Impact Rate: 1/5

Likelihood Rate: 5/5

Exploitability: Independent

Complexity: Simple

Severity: Info

Recommendations

Remediation:

Use unchecked blocks for the safe arithmetic in `claim()`, `claimAll()`, `decreaseAllocation()`, and `decreaseAllocationBatch()`.

Resolution:

Fixed in commit `621c8df`.

Aforementioned functions now use unchecked blocks, and `_owner` is cached in `transferOwnership()` function.

Disclaimers

Hacken Disclaimer

The smart contracts given for audit have been analyzed based on best industry practices at the time of the writing of this report, with cybersecurity vulnerabilities and issues in smart contract source code, the details of which are disclosed in this report (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

The report contains no statements or warranties on the identification of all vulnerabilities and security of the code. The report covers the code submitted and reviewed, so it may not be relevant after any modifications. Do not consider this report as a final and sufficient assessment regarding the utility and safety of the code, bug-free status, or any other contract statements.

While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only — we recommend proceeding with several independent audits and a public bug bounty program to ensure the security of smart contracts.

English is the original language of the report. The Consultant is not responsible for the correctness of the translated versions.

As part of Hacken's ongoing quality assurance process, we may conduct re-audits of select projects. These re-audits are performed independently from the original audit and are intended solely for internal quality control and improvement. Updated reports resulting from such re-audits will be shared privately with the respective clients and may be published on the Hacken website only with their explicit consent.

The sole authoritative source for finalized and most up-to-date versions of all reports remains the Audits section at <https://hacken.io/audits/>.

Technical Disclaimer

Smart contracts are deployed and executed on a blockchain platform. The platform, its programming language, and other software related to the smart contract can have vulnerabilities that can lead to hacks. Thus, the Consultant cannot guarantee the explicit security of the audited smart contracts.

Appendix 1. Definitions

Severities

When auditing smart contracts, Hacken is using a risk-based approach that considers **Likelihood**, **Impact**, **Exploitability** and **Complexity** metrics to evaluate findings and score severities.

Reference on how risk scoring is done is available through the repository in our Github organization:

[hknio/severity-formula](https://github.com/hacken/severity-formula)

Severity	Description
Critical	Critical vulnerabilities are usually straightforward to exploit and can lead to the loss of user funds or contract state manipulation.
High	High vulnerabilities are usually harder to exploit, requiring specific conditions, or have a more limited scope, but can still lead to the loss of user funds or contract state manipulation.
Medium	Medium vulnerabilities are usually limited to state manipulations and, in most cases, cannot lead to asset loss. Contradictions and requirements violations. Major deviations from best practices are also in this category.
Low	Major deviations from best practices or major Gas inefficiency. These issues will not have a significant impact on code execution.

Potential Risks

The "Potential Risks" section identifies issues that are not direct security vulnerabilities but could still affect the project's performance, reliability, or user trust. These risks arise from design choices, architectural decisions, or operational practices that, while not immediately exploitable, may lead to problems under certain conditions. Additionally, potential risks can impact the quality of the audit itself, as they may involve external factors or components beyond the scope of the audit, leading to incomplete assessments or oversight of key areas. This section aims to provide a broader perspective on factors that could affect the project's long-term security, functionality, and the comprehensiveness of the audit findings.

Appendix 2. Scope

The scope of the project includes the following smart contracts from the provided repository:

Scope Details	
Repository	https://github.com/Haycen/yield-claim-smart-contract
Commit	6e5135d8c6781d150c5407a65fcce36b2a31775a
Final	621c8df0d65a2a656c2ef5cbd2e993d9b777e322
Whitepaper	N/A
Requirements	https://github.com/Haycen/yield-claim-smart-contract/blob/master/documentation/AUDIT_BRIEF.md
Technical Requirements	N/A

Asset	Type
contracts/YieldClaim.sol [https://github.com/Haycen/yield-claim-smart-contract]	Smart Contract

Appendix 3. Additional Valuables

Additional Recommendations

The smart contracts in the scope of this audit could benefit from the introduction of automatic emergency actions for critical activities, such as unauthorized operations like ownership changes or proxy upgrades, as well as unexpected fund manipulations, including large withdrawals or minting events. Adding such mechanisms would enable the protocol to react automatically to unusual activity, ensuring that the contract remains secure and functions as intended.

To improve functionality, these emergency actions could be designed to trigger under specific conditions, such as:

- Detecting changes to ownership or critical permissions.
- Monitoring large or unexpected transactions and minting events.
- Pausing operations when irregularities are identified.

These enhancements would provide an added layer of security, making the contract more robust and better equipped to handle unexpected situations while maintaining smooth operations.

Frameworks and Methodologies

This security assessment was conducted in alignment with recognised penetration testing standards, methodologies and guidelines, including the [NIST SP 800-115 – Technical Guide to Information Security Testing and Assessment](#), and the [Penetration Testing Execution Standard \(PTES\)](#). These assets provide a structured foundation for planning, executing, and documenting technical evaluations such as vulnerability assessments, exploitation activities, and security code reviews. Hacken's internal penetration testing methodology extends these principles to Web2 and Web3 environments to ensure consistency, repeatability, and verifiable outcomes.